

Übungsblatt ÜE-08

Ausgabe: 07.01.2026

Abgabe: 24.01.2026

12:00 Uhr

Der Lauf der Natur – Team-Blatt (2 Wochen)

Hinweise:

- Dieses Aufgabenblatt ist in **Zweierteams** zu bearbeiten. Teamarbeit ist Teil des Lerninhaltes. Die Teams sollten aus einem Tutorium stammen. Die Tutoren und Tutorinnen sind angehalten in den Tutorien die Teambildung zu unterstützen. Die finale Bewertung durch die Tutoren und Tutorinnen erfolgt dabei erst nach einem erfolgreichem Code Review. Dort wird geprüft, ob von allen Teammitgliedern der abgegebene Inhalt auch verstanden wurde. ... es geht also nicht darum die Arbeit einfach aufzuteilen! **Jeder muss alles verstehen!**

Wer keinen Partner findet, muss dies spätestens bis zum 17.01. an seine Tutorin/Tutor melden und wird dann zugeteilt. Einzelabgaben sind nur in Notfällen (Teampartner bricht weg) möglich. In diesem Fall ist unverzüglich der Tutor oder die Tutorin zu informieren. Ansonsten werden Einzelabgaben mit Null (0) Punkten bewertet.

- **Pro Team nur eine Abgabe in Moodle!**
- *Sie dürfen für dieses Blatt alles nutzen, jedoch ist es nicht zielführend automatisch generierte Lösungen (z.B. über ein LLM) einfach zu kopieren und hochzuladen. **Quellen sind anzugeben, also auch ob ein LLM bei der Erstellung unterstützt hat!***
- *Dieses Blatt ist in 2er Teams zu lösen! Mit der `__author__` Angabe (siehe nächster Punkt) bestätigen Sie, dass die Abgabe Ihr Werk ist. Die Abgaben werden auch auf **Plagiate** geprüft. Sollten wir identische Abgaben finden, erhält keiner der abgebenden Punkte.*
- *Achten Sie darauf die Variable `__author__` in allen Quellcode Dateien (.py) korrekt zu setzen (am Anfang des Quellcodes – Teamabgabe!):*
`__author__ = "<Matr-Nr_1>, <Nachname_1>, <Matr-Nr_2>, <Nachname_2>"`
Beispiel: `__author__ = "1234568, Kaschube, 1234567, Tolle"`
... Leerstellen vor und nach dem „=“ und Leerstelle nach dem Komma beachten, sowie keine spitzen Klammern verwenden.
- *Digitale Abgaben, die nicht im Format **.pdf** oder **.txt** für Texte oder **.py** für Code erfolgen, werden nicht überprüft. Bei Abgaben mehrerer Dateien müssen diese als **.zip** zusammengefasst werden.*
- *Sollten Aufgabenstellungen nicht eindeutig sein, bitte durch Ihre eigenen Annahmen ergänzen und diese dokumentieren (z.B. in einem Kommentar).*
- *Achten Sie auch auf Vorgaben Ihrer Tutorin/Tutors. Hierfür auch in das Gruppenforum im Moodle-Kurs von EPR schauen!*

- Beim Programmieren und Kommentieren halten Sie sich bitte an die Regeln im Programmierhandbuch (Style Guide), siehe Moodle-Kurs. Im Zweifelsfall gilt PEP 8. ... möglichst selbst mit `pycodestyle` mal prüfen. Nicht einhalten der Konventionen führt auch zu Punktabzug.
- Doc-Strings sind für jedes Modul, Funktion, Klasse, Methode, ... zwingend anzugeben. Weiterhin ist jede Funktion/Methode mit mindestens drei Testfällen im Code zu versehen. Was in diesem Blatt eh Teil der Aufgabenstellung ist und in Form von Doctests umzusetzen ist.
- Achten Sie darauf, dass Ihr Code **lauffähig** ist. Bereiche/Funktionalitäten, die nicht lauffähig sind, werden standardmäßig mit 0 Punkten bewertet.
- Datei- und Ordnernamen sollen keine Umlaute, diakritische oder Sonderzeichen, mit Ausnahme des Unterstrichs, enthalten!
- Ihre Lösung muss ohne Installation von nicht in Python 3.1x enthaltenen Standard-Bibliotheken auskommen. Die Korrektur und Ausführung durch die Tutoren muss also **ohne** ein „pip install ...“ auskommen.

Für dieses Blatt können bis zu **30 Punkte** erreicht werden. Neben der Erfüllung der Aufgabenstellung durch entsprechender Abgabe im Moodle-Kurs, ist ein Code Review bei Ihrer Tutorin/Tutor notwendig. → **kein code review = 0 Punkte**
 Im Code Review sollen Sie Ihre Lösung vorstellen. Dies sollte vorbereitet werden und ist Teil der Aufgabe, sprich wird ebenfalls mit bewertet. Bereiche, die im Codereview nicht erklärt werden können, werden ebenfalls mit 0 Punkten bewertet.

Aufgabe – Ökosystem- der Lauf der Natur

Hier soll ein Ökosystem mithilfe von Objektorientierung implementiert werden. Das Ökosystem ist in einem Habitat. In diesem Habitat sollen verschiedene Lebewesen wie Pflanzenfresser, Allesfresser und Fleischfresser und Pflanzen leben.

- Eine Pflanze braucht Platz in dem Habitat. Bei der Erstellung eines Habitats soll direkt die Größe und weitere Eigenschaften initialisiert werden.
- Auch die Anzahl der Lebewesen, die in dem Habitat (kann man sich wie eine Insel vorstellen, der Platz ist also begrenzt) leben können, hängt von der Nahrung ab, die im Habitat zu finden ist. Dabei können Lebewesen andere Lebewesen oder Pflanzen fressen oder auch sterben.

Hier ein paar Grundregeln:

- Das Ökosystem soll in Runden simuliert werden (das macht es einfacher). In jeder Runde sollen Lebewesen wachsen, sich vermehren und sich fortpflanzen. Tiere müssen weiterhin fressen, um nicht zu verhungern. Lebewesen können sterben, z.B. durch Krankheiten, Verletzungen, weil Sie gefressen werden, Nahrungsmangel oder einfach durch hohes Alter.
- In einem Habitat sollen Lebewesen und Pflanzen leben. Dabei benötigen Pflanzen einen gewissen Platz, um sich zu vergrößern, um ihr volles Potenzial zu erreichen.
- In einem Habitat soll es mindestens drei (3) verschiedene Arten von Pflanzen geben. Jede Pflanzenart hat seine Vorteile und Einschränkungen. Eine Pflanze hat eine minimale und eine maximale Größe. Sollte das Habitat nicht mehr genug Platz haben, kann eine Pflanze nicht weiterwachsen. Wird von einer Pflanze mehr gefressen als ihre minimale Größe definiert, geht sie ein (stirbt).
- Es leben mindestens drei (3) verschiedenen Lebewesen Klassen (Pflanzenfresser, Allesfresser und Fleischfresser) in dem Habitat.
 Jedes Lebewesen sucht sich Futter, dass mal mehr oder weniger erfolgreich ist. Wenn das Lebewesen zu wenig Nahrung findet, stirbt es. Fleischfresser oder auch Allesfresser können auch jagen. Je nach Situation ist eine Jagd erfolgreich oder nicht, manchmal auch gefährlich.

Weitere Eigenschaften:

- Jede Runde entspricht eine gewissen Zeiteinheit, die Ihr festlegt (abhängig von der Zeiteinheit und dem Lebewesen müssen also nicht immer alle Aktivitäten wie z.B. Vermehren ausgeführt werden).
- In jeder Runde regenerieren sich die Pflanzen zu einem gewissen Teil. Das hängt von Pflanze zu Pflanze ab. Optional können auch Jahreszeiten definiert werden, was auch Einfluss auf das Pflanzenwachstum haben kann.
- Jedes Lebewesen muss über einen gewissen Zeitraum/Runden genug zum Essen finden, das kann von Lebewesen zu Lebewesen variieren (ein Kolibri braucht regelmäßig Nektar, eine Kakerlake kann auch längere Zeit ohne Essen auskommen). Ist das nicht der Fall wird es schwächer und stirbt am Ende.

Anforderungen:

- Erstellen Sie ein UML-Klassendiagramm, dass den Aufbau und die Interaktion der Klassen untereinander veranschaulicht (das UML-Klassendiagramm MUSS mit einem Tool erstellt (z.B. Umbrello) werden und darf nicht händisch gezeichnet sein).
- Übertragen sie das UML-Diagramm in Python. Beachten Sie dort insbesondere auch die Assoziationen, Multiplizitäten und wer von wem erben soll. All die Eigenschaften, Einschränkungen und Regeln sollen über die Methoden und Attribute der verschiedenen Klassen eingehalten werden.
- Erstellen Sie mindestens 5 weitere Regeln, die die oberen Regeln erweitern oder teilweise aufheben. Zum Beispiel: Der Eukalyptus kann nur von Koalabären gegessen werden. Was passiert, wenn der Platz im Habitat voll ist und keine weiteren Pflanzen wachsen können? Es kann auch Pflanzen geben, die auf Bäumen wachsen und so keinen zusätzlichen Platz des Habitats benötigen. Anstatt zu sterben, können Tiere auch in einen Ruhezustand (Winterschlaf, Winterruhe) übergehen, Stichwort Jahreszeiten. Halten sie alle zusätzlichen Regeln in einer Dokumentation fest (siehe Dokumentation unten)!
Hinweis: Wenn möglich sollten die Regeln durch Überschreiben der entsprechenden Methoden der Oberklasse umgesetzt werden.
- Verschiedene Aspekte (mindestens 2) sollen vom Zufall abhängen (random). Hiervon kann z.B. der Jagderfolg oder auch die Stärke des Wachstums der Pflanzen (quasi als Wettereinfluss) abhängen.
- Neben der Implementierung soll auch eine **Dokumentation** (extra PDF) erstellt werden: Dort sollen die Regeln erklärt werden und auch die Benutzerführung (Benutzerhandbuch).
- Denkt weiterhin an Docstrings, Testfälle und Einhalten des Style-Guides.

User Interface:

- Erstellen Sie ein UI auf Konsolenebene, das den Nutzer nach seiner Startbelegung erfragt. Danach soll nach Aktivierung des Ökosystems kein weiterer Eingriff in das Ökosystem mehr notwendig sein (zum Debuggen oder Testen darf dies aber natürlich vorhanden sein).
- Nach der Aktivierung soll es die Möglichkeit geben das Ökosystem zu pausieren bzw. Runden zu simulieren. ... und den daraus resultierenden Zustand anzuzeigen. Überlegen Sie sich entsprechend, wie dem Nutzer eine übersichtliche Darstellung darüber geben werden kann, in welchem Zustand sich das Ökosystem nach einer Runde befindet. Implementieren Sie verschiedene Modi, die die Geschwindigkeit anpasst, so dass mehrere Runden laufen, ohne das eine Ausgabe oder anderweitige Interaktion erfolgt.
- Beachten Sie dabei, dass Sie aussagekräftige Eingabeaufforderungen verwenden und das die Eingabe robust ist, also das das Programm bei einer Falscheingabe nicht abstürzt.

Bemerkung: Es gibt viele Optionen und auch Freiheiten bei der Umsetzung. Starten Sie erst einmal einfach. ... schwieriger geht immer! Ansonsten hoffen wir, dass es auch ein wenig Spaß macht ;-)